

MCA : Assignment 1B

1. What is the difference between combinational and sequential logic circuits?

- **Combinational Logic Circuits:** In these circuits, the output depends only on the current input. There is no memory element in these circuits, so the output changes immediately as the input changes.
 - Example: Adders, multiplexers.
- **Sequential Logic Circuits:** In these circuits, the output depends on both the current input and the previous inputs (stored memory). They have memory elements, which means the output can change based on the history of inputs.
 - Example: Flip-flops, counters.

2. How does a flip-flop function in sequential logic?

- A flip-flop is a basic memory element in sequential circuits. It stores one bit of information and has two stable states (0 or 1).
- It changes its state based on input signals (like clock signals) and holds its state until a change is triggered.
 - Example: A D flip-flop stores data on the rising or falling edge of a clock signal.

3. Can you explain how a multiplexer works in combinational logic?

- A multiplexer (MUX) is a combinational circuit that selects one of many inputs and passes it to the output based on a set of selection lines.
- The MUX has multiple data inputs, but only one output, and the selected input is passed based on the control (selection) lines.
 - Example: 2-to-1 MUX has 2 data inputs (I_0 , I_1) and 1 output (Y), with a selection line (S) deciding which input gets passed to the output.

4. What are the main types of registers in a computer system?

- General-purpose registers: These are used to hold temporary data or intermediate results during execution. Examples: AX, BX, CX in x86 architecture.
- Special-purpose registers: These are designed to perform specific tasks related to control or status. Examples include:
 - Program Counter (PC): Holds the address of the next instruction to be executed.
 - Accumulator (AC): Stores the result of arithmetic operations.
 - Status Register (SR): Stores flags that indicate the status of the CPU (e.g., zero, carry, etc.).

5. How does a CPU utilize registers for instruction execution?

- The CPU uses registers to store temporary data and control information during the execution of instructions.
 - Instruction Register (IR): Holds the current instruction being executed.
 - Program Counter (PC): Keeps track of the memory address of the next instruction.
 - Data registers: Hold operands or results of operations.
- The CPU fetches instructions, decodes them, and executes them, using the data stored in registers for fast access.

6. What is the role of the accumulator register in arithmetic operations?

- The accumulator is a special-purpose register used to store the intermediate or final result of arithmetic and logic operations.
- It works closely with the ALU (Arithmetic Logic Unit).
- Example: If CPU adds two numbers, one number comes from memory, and the other is already in the accumulator.
- It helps in speeding up calculations by reducing memory access.

7. Can you explain the difference between a general-purpose register and a special-purpose register?

- General-Purpose Registers:
 - Used for temporary storage of data and intermediate results.
 - Programmers can use them freely.
 - Example: AX, BX, CX, DX in x86 processors.
- Special-Purpose Registers:
 - Used for specific control tasks in the CPU.
 - Not freely usable by programmers.
 - Examples:
 - Program Counter (PC): Points to next instruction.
 - Instruction Register (IR): Holds current instruction.
 - Stack Pointer (SP): Points to top of the stack.

8. What is meant by timing control in digital circuits?

- Timing control ensures that different parts of a digital system work in proper sequence and at the correct time.
- This is usually controlled by a clock signal, which provides regular timing pulses.
- Timing control is essential for synchronizing operations like data transfer, reading/writing memory, or executing instructions.

9. How do clock cycles impact the performance of sequential circuits?

- A clock cycle is a single pulse of the clock signal that controls when data is moved or operations are performed.
- In sequential circuits, operations happen on clock edges (rising or falling).
- More clock cycles to perform a task = slower performance.
- Fewer clock cycles = faster execution.
- A higher clock speed (in Hz) means the circuit can perform more operations per second.

10. What is the role of a control unit in a CPU's operation?

- The Control Unit (CU) acts like the brain within the CPU's brain.
- It directs the flow of data between the CPU, memory, and input/output devices.
- Main functions:
 - Fetch instructions from memory.
 - Decode the instruction.
 - Send control signals to other units (ALU, memory, registers).
- It ensures that everything happens in the right order and timing.

11. Can you describe the phases of an instruction cycle in a CPU?

The instruction cycle has 3 main phases:

1. Fetch:
 - CPU gets (fetches) the instruction from memory using the Program Counter (PC).
 - The instruction is loaded into the Instruction Register (IR).
2. Decode:
 - The Control Unit decodes the instruction to understand what needs to be done.
 - It identifies the operation and the operands (data to be used).
3. Execute:
 - The CPU performs the required operation (like add, load, store).
 - The result is stored in the proper register or memory.

Optional 4th phase: 4. Store (Write-back):

- Final result is written back to memory or a register.

12. What are memory reference instructions, and how are they used in a processor?

- Memory reference instructions are commands that involve accessing memory — either reading from it or writing to it.
- Common types include:

- LOAD (Read): Brings data from memory into a register.
- STORE (Write): Saves data from a register into memory.
- These instructions help the CPU interact with RAM, which is essential for real-world programs.

13. How does a load instruction differ from a store instruction?

- LOAD Instruction:
 - Transfers data from memory to a register.
 - Example: LOAD R1, 1000 → load data from memory address 1000 into register R1.
- STORE Instruction:
 - Transfers data from a register to memory.
 - Example: STORE R1, 1000 → store value from register R1 into memory address 1000.
- In short:
 - LOAD = Read from memory
 - STORE = Write to memory

14. What is the purpose of an I/O interruption in a computer system?

- I/O Interruption (or interrupt) is used when an input/output device needs attention from the CPU.
- Instead of constantly checking the device, the CPU continues its work and responds only when interrupted.
- Purpose:
 - To save time and increase efficiency.
 - CPU handles the I/O operation only when needed, not continuously.

15. How does an interrupt request (IRQ) affect the flow of execution in a processor?

- When an IRQ (Interrupt Request) occurs:
 1. CPU pauses the current task.
 2. It saves the current state (like program counter and register values).
 3. CPU then jumps to the interrupt handler (special routine to handle the interrupt).
 4. After handling, it returns to the original task and resumes execution.
- This allows the CPU to respond quickly to important events like keyboard input, mouse movement, or disk read.

16. How does a full adder circuit differ from a half adder circuit?

- **Half Adder:**
 - Adds two binary digits.
 - Outputs: Sum and Carry.
 - No input for previous carry (cannot handle carry-in).
 - Useful for simple addition (like the first bit).
- **Full Adder:**
 - Adds three inputs: two binary digits and a carry-in from the previous stage.
 - Outputs: Sum and Carry-out.
 - Used in cascaded addition (like multi-bit binary addition).

Key Difference: Full adder handles carry-in; half adder does not.

17. What is the role of a subtractor circuit in digital systems?

- A subtractor circuit performs binary subtraction.
- It calculates the difference between two binary numbers.
- Two main types:
 - Half Subtractor: Subtracts two bits (no borrow input).
 - Full Subtractor: Subtracts two bits and also handles a borrow-in.

- Used in arithmetic operations inside ALU (like subtract, compare).

18. How does Booth's multiplication algorithm optimize the multiplication process in binary numbers?

- Booth's algorithm improves binary multiplication by:
 - Reducing the number of add/subtract operations.
 - Grouping consecutive 1s in the multiplier to speed up the process.
- It works well for signed binary numbers (like negative numbers).
- More efficient than regular shift-and-add method for certain patterns.

19. Can you explain the basic steps involved in the Booth multiplication algorithm?

Here are the simplified steps:

1. Initialize:
 - Set multiplier, multiplicand, and accumulator.
 - Add an extra bit to the multiplier for comparison.
2. Repeat for number of bits:
 - Check the last two bits of multiplier (current + previous bit).
 - If 10: Subtract multiplicand from accumulator.
 - If 01: Add multiplicand to accumulator.
 - If 00 or 11: Do nothing.
 - Arithmetic Right Shift of accumulator and multiplier together.
3. After all cycles:
 - Final result is in accumulator + multiplier.

20. What is multi-core architecture, and how does it improve the performance of a computer system?

- Multi-core architecture means a CPU has more than one core (processing unit) on a single chip.
- Each core can perform tasks independently or together, allowing parallel processing.
- Benefits:
 - Better multitasking (run more apps at once).
 - Faster performance for programs that support parallel execution.
 - Lower power usage per task compared to single-core running faster.

